

Computer Architectures

Exam of 17.05.2023 – part II

Question 1

The instruction set of ARM v7-M contains an instruction for the signed division with 32-bit operands: `SDIV Rd, Rn, Rm` where `Rd` is the quotient, `Rn` is the dividend, and `Rm` is the divisor. You have to write the `SDIV64` subroutine in ARM assembly language, which computes the signed division between a 64-bit dividend and a 32-bit divisor, and returns a 32-bit quotient. The subroutine receives in input:

- `U`: the upper half of the dividend
- `L`: the lower half of the dividend
- `D`: the divisor

The following algorithm is used to compute the quotient `Q`:

1. if the dividend and the divisor have the same sign:
2. `flag = true`
3. else
4. `flag = false`
5. if the dividend is negative:
6. `dividend = opposite(dividend)`
7. if the divisor is negative:
8. `divisor = opposite(divisor)`
9. for `i = 0` to `31`:
10. double the dividend: `U = 2U, L = 2L`
11. if `U >= D`:
12. `U = U - D`
13. set `i`-th bit of `Q` to 1
14. else:
15. set `i`-th bit of `Q` to 0
16. if `flag == false`
17. `Q = opposite(Q)`
18. return `Q`

You can not use multiplication. In particular:

- At points 6, 8, and 17, you have to compute the two's complement (instead of multiplying by -1) to get the opposite of the number. Two's complement is achieved by inverting all bits and adding 1 to the entire inverted number.
- At point 10, you have to use the left shift (instead of multiplication) to double the dividend. The most significant bit of `L` becomes the least significant bit of `U`.

At points 13 and 15, the bits are counted from left to right. E.g., bit 0 is the most significant one.

An example is given here considering 4-bit registers (the dividend is expressed with 8 bits). You can easily extend it to the 32-bit case.

Example: `U = 1101, L = 1010, D = 1011`

dividend and the divisor have the same sign => `flag = true`

dividend is negative => `U = 0010, L = 0110`

divisor is negative => `D = 0101`

Iteration `i = 0`

`U = 0100, L = 1100`

Computer Architectures

Exam of 17.05.2023 – part II

$U < D \rightarrow$ bit 0 of Q is 0

Iteration i = 1

$U = 1001, L = 1000$

$U \geq D \rightarrow U = 1001 - 0101 = 0100$ bit 1 of Q is 1

Iteration i = 2

$U = 1001, L = 0000$

$U \geq D \rightarrow U = 1001 - 0101 = 0100$ bit 2 of Q is 1

Iteration i = 3

$U = 1000, L = 0000$

$U \geq D \rightarrow U = 1000 - 0101 = 0011$ bit 3 of Q is 1

The quotient is $Q = 0111$

flag = true $\Rightarrow Q = 0111$

Computer Architectures

Exam of 17.05.2023 – part II

Question 2

Write the `SDIV64S` subroutine, which extends the subroutine of the previous exercise by setting the flags in the Program Status Register (PSR). The bits in PSR are:

- Bit 31 (most significant bit): negative flag
- Bit 30: zero flag
- Bit 29: carry flag
- Bit 28: overflow flag
- Bit 27: sticky saturation flag
- Bit 26, 25, 15-10: interrupt-continuable instruction (ICI) bits, IF-THEN instruction status bit
- Bit 24: Thumb state
- Bit 8-0: exception number.

At the end of the procedure, the flags have to be:

- negative flag = first bit of the quotient
- zero flag = 1 if the quotient is zero; zero flag = 0 if the quotient is different from 0
- carry flag = 0
- overflow flag = 1 if the quotient can not be stored in 32 bits; overflow flag = 0 otherwise.

The procedure does not change the other bits in PSR.

The value of the overflow flag can be determined at the beginning of the subroutine by comparing:

- $|U|$: the absolute value of the upper half of the dividend U (i.e., U if U is positive, $-U$ if U is negative)
- $|D|/2$: half of the absolute value of the divisor D .

If $|D|/2 > |U|$, then the overflow flag is 0, otherwise is 1.

Note: you do not have to copy duplicated code from the previous exercise. Write new instructions; you can write comments to indicate duplicated section of code.